TUGAS PERTEMUAN: 6

DATABASE

NAMA/NIM	:	Rahman Mauliddin Anwar (2618903)
KELAS	:	D
PENGAJAR	:	STEFANUS NATHANAEL HERMANTO PUTRA (2518004)

6.1 Tujuan

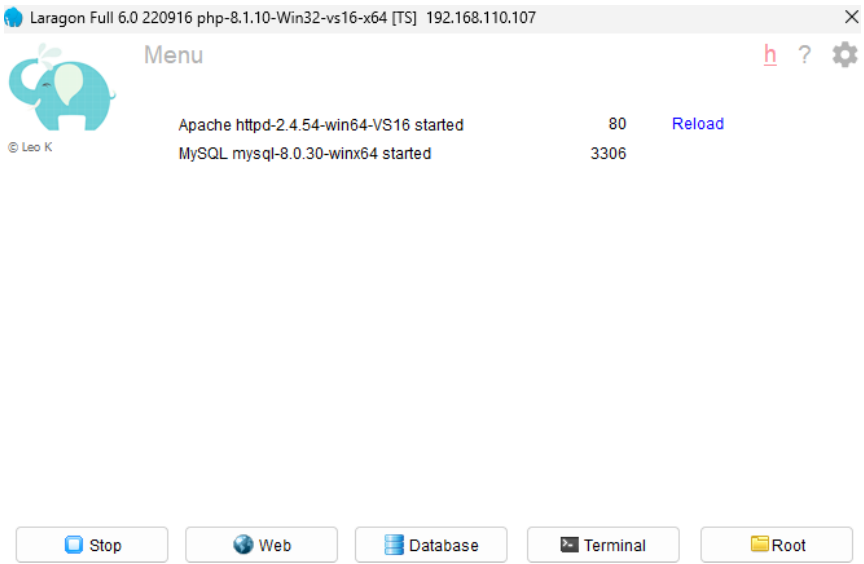
1. Praktikan mampu memahami dasar-dasar Database.
2. Praktikan mampu membuat database dan tabel menggunakan perintah SQL.
3. Praktikan mampu mengimplementasikan perintah *Insert, Update, Delete, Select*, dan *Join*.

6.2 Alat dan bahan

1. Laptop
2. *Laragon*
3. Modul Praktikum Web 2026

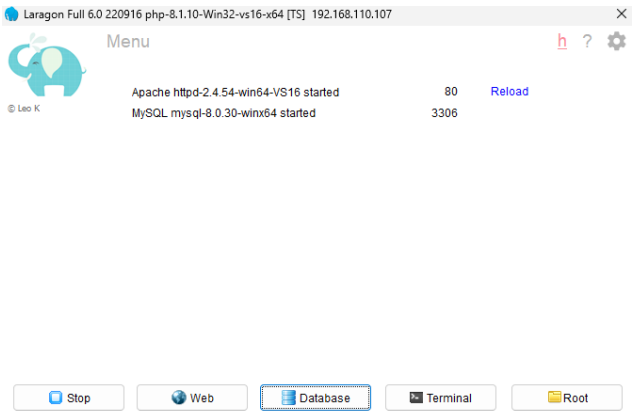
6.3 Langkah-langkah

1. Buka *Laragon*, Kemudian aktifkan *laragon* dengan klik tombol *StartAll*.



Gambar 6. 1 Tampilan *Laragon* ketika di *StartAll*

2. Lalu, Tekan tombol *Database*.



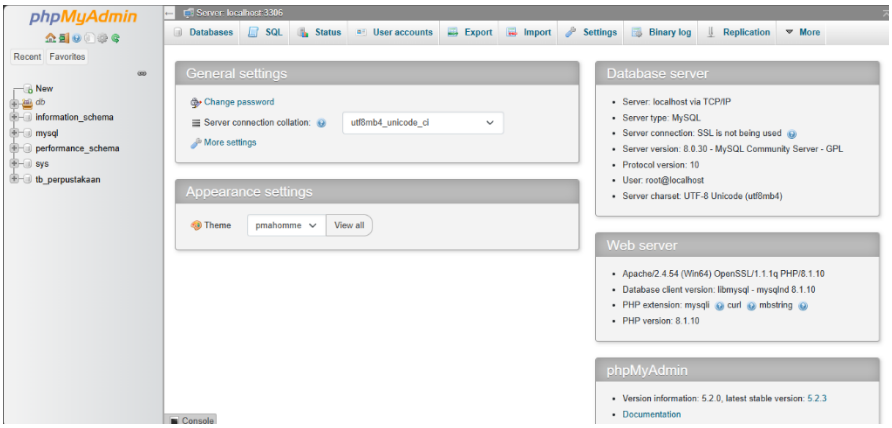
Gambar 6. 2 Tampilan Klik Tombol *Database*

3. klik tombol *Database* untuk membuka *phpMyAdmin*. *Login* dengan *username root.*



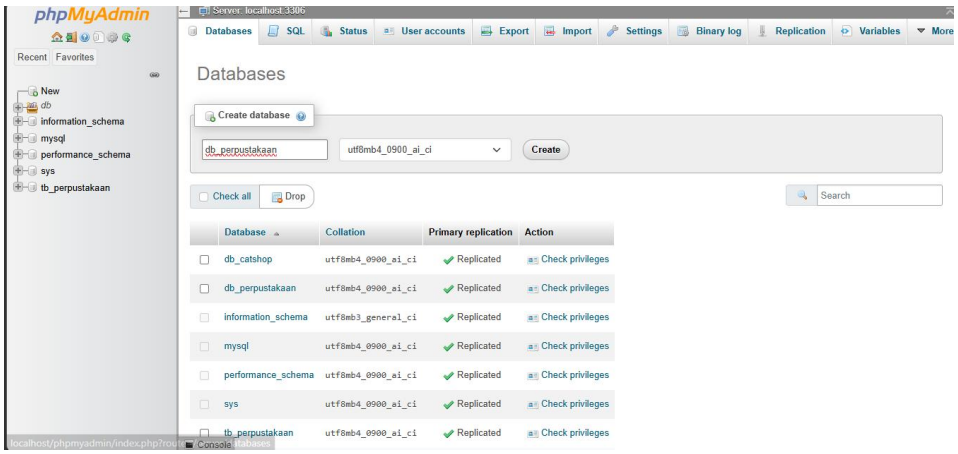
Gambar 6. 3 Tampilan *Login di phpMyAdmin*

4. Pada halaman ini, Anda dapat melihat berbagai menu untuk mengelola *database*, seperti membuat *database* baru, membuat tabel, menambahkan data, mengubah data, dan menghapus data.



Gambar 6. 4 Tampilan Halaman berbagai menu *Database*

5. Lalu, buat *database* baru dengan menekan teks *New* pada bagian kiri layar, lalu mengisi nama *database* yang akan dibuat. Setelah itu klik tombol *Create*.



Gambar 6. 5 Tampilan Membuat *Database* baru

6. Setelah membuat *database* baru, kamudian kita buat 3 tabel dengan *field* yang sudah di tentukan.

   Source Code :

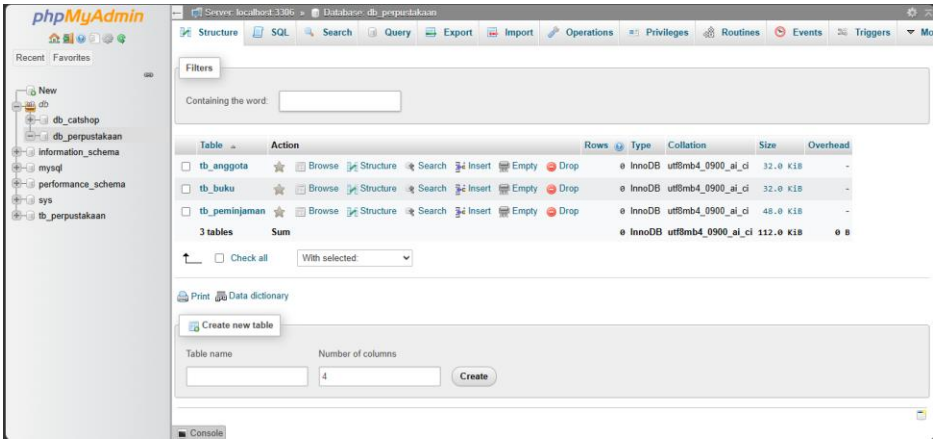
```
USE db_perpustakaan;
CREATE TABLE tb_anggota (
  id_anggota INT(11) PRIMARY KEY AUTO_INCREMENT,
  nama VARCHAR(50) NOT NULL,
  email VARCHAR(50) UNIQUE,
  nomorhp VARCHAR(15),
  alamat TEXT,
  tgl_daftar DATE
);

CREATE TABLE tb_buku (
  id_buku INT(11) PRIMARY KEY AUTO_INCREMENT,
  kode_buku VARCHAR(10) UNIQUE,
  judul VARCHAR(100) NOT NULL,
  pengarang VARCHAR(50),
  tahun INT(4),
  stok INT(11) DEFAULT 0
);

CREATE TABLE tb_peminjaman (
  id_pinjam INT(11) PRIMARY KEY AUTO_INCREMENT,
  id_anggota INT(11),
  id_buku INT(11),
  tgl_pinjam DATE NOT NULL,
  tgl_kembali DATE NULL,
  status VARCHAR(20) DEFAULT 'Dipinjam',
  FOREIGN KEY (id_anggota) REFERENCES
tb_anggota(id_anggota),
```

```
  FOREIGN KEY (id_buku) REFERENCES tb_buku(id_buku)
);
```

Hasil :



Gambar 6. 6 Tampilan Membuat tabel dengan *field* yang sudah di tentukan.

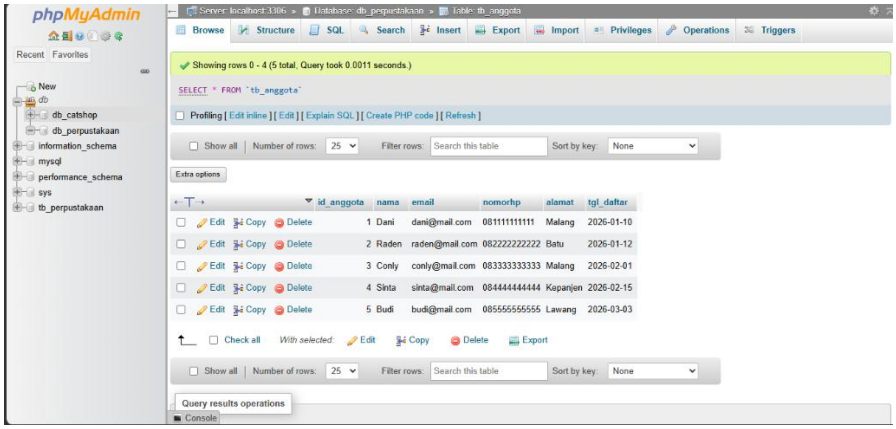
7. Isi tabel tb_anggota dengan 5 data menggunakan INSERT satu per satu.

   Source Code :

```
INSERT INTO tb_anggota (nama, email, nomorhp, alamat,
tgl_daftar)
VALUES ('Dani', 'dani@mail.com', '081111111111', 'Malang',
'2026-01-10');
INSERT INTO tb_anggota (nama, email, nomorhp, alamat,
tgl_daftar)
VALUES ('Raden', 'raden@mail.com', '082222222222', 'Batu',
'2026-01-12');
INSERT INTO tb_anggota (nama, email, nomorhp, alamat,
tgl_daftar)
VALUES ('Conly', 'conly@mail.com', '083333333333',
'Malang', '2026-02-01');
INSERT INTO tb_anggota (nama, email, nomorhp, alamat,
tgl_daftar)
VALUES ('Sinta', 'sinta@mail.com', '084444444444',
'Kepanjen', '2026-02-15');
INSERT INTO tb_anggota (nama, email, nomorhp, alamat,
tgl_daftar)
VALUES ('Budi', 'budi@mail.com', '085555555555', 'Lawang',
'2026-03-03');

SELECT * FROM tb_anggota;
```

Hasil :



Gambar 6. 7 Tampilan Isi tabel tb_anggota

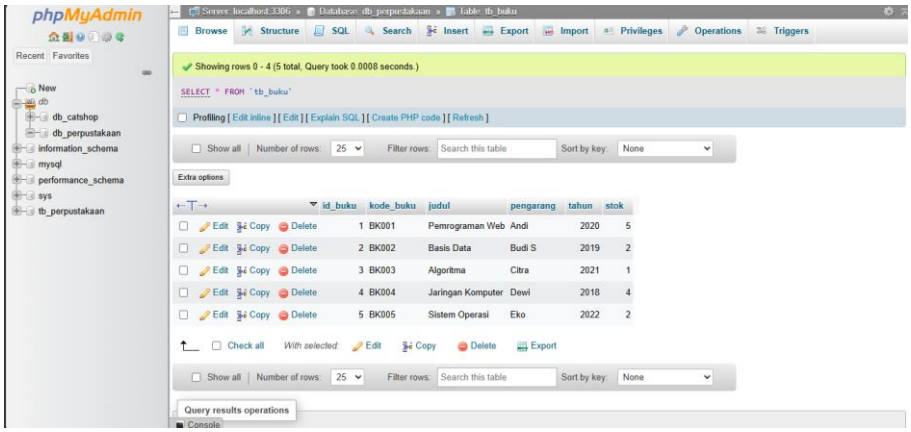
8. Setelah itu, Isi tabel tb_buku dengan 5 data menggunakan satu perintah INSERT untuk banyak data.

Source Code :

```
INSERT INTO tb_buku (kode_buku, judul, pengarang, tahun,
stok) VALUES
('BK001', 'Pemrograman Web', 'Andi', 2020, 5),
('BK002', 'Basis Data', 'Budi S', 2019, 2),
('BK003', 'Algoritma', 'Citra', 2021, 1),
('BK004', 'Jaringan Komputer', 'Dewi', 2018, 4),
('BK005', 'Sistem Operasi', 'Eko', 2022, 2);

SELECT * FROM tb_buku;
```

Hasil :



Gambar 6. 8 Tampilan Isi tabel tb_buku

9. Kemudian, Isi tabel tb_peminjaman dengan 5 data.

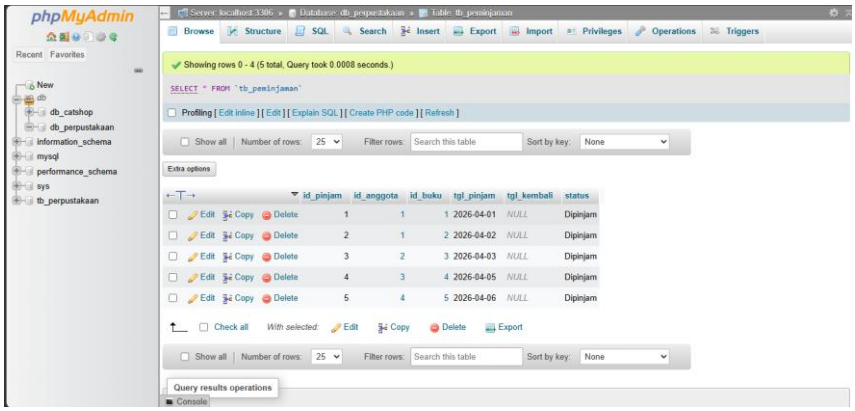
Source Code :

```
INSERT INTO tb_peminjaman (id_anggota, id_buku, tgl_pinjam,
tgl_kembali, status) VALUES
(1, 1, '2026-04-01', NULL, 'Dipinjam'),
```

```
(1, 2, '2026-04-02', NULL, 'Dipinjam'),
(2, 3, '2026-04-03', NULL, 'Dipinjam'),
(3, 4, '2026-04-05', NULL, 'Dipinjam'),
(4, 5, '2026-04-06', NULL, 'Dipinjam');

SELECT * FROM tb_peminjaman;
```

Hasil :



Gambar 6. 9 Tampilan Isi tabel tb_peminjaman

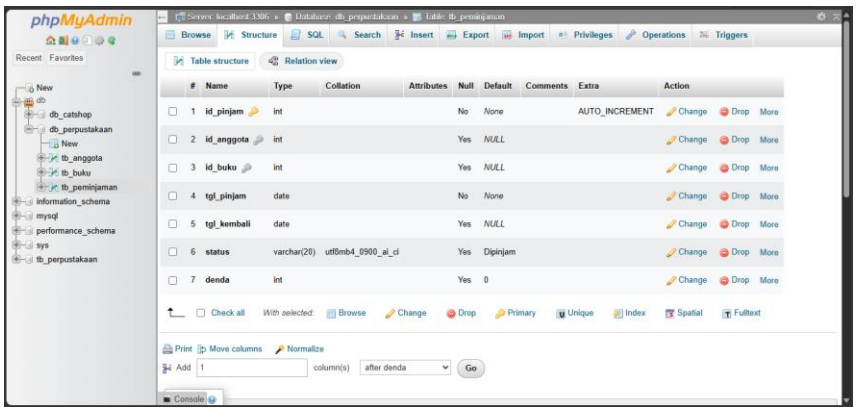
10. Tambahkan field denda (INT, default 0) setelah field status pada tb_peminjaman.

Source Code :

```
ALTER TABLE tb_peminjaman ADD denda INT(11) DEFAULT 0 AFTER
status;

DESCRIBE tb_peminjaman;
```

Hasil:



Gambar 6. 10 Struktur tb_peminjaman setelah penambahan *field* denda

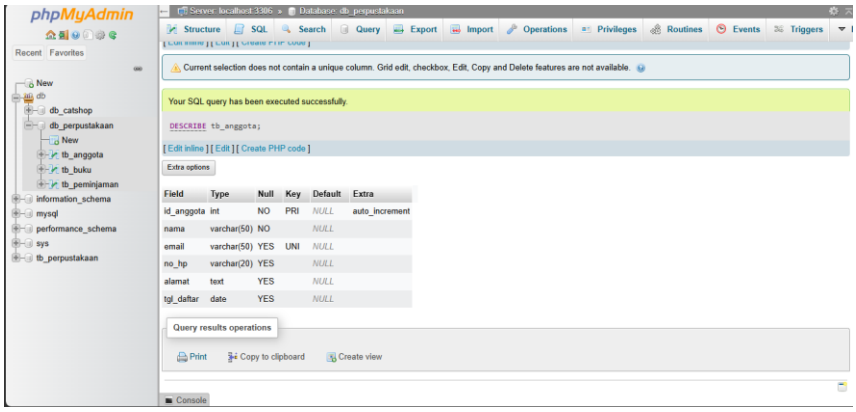
11. Ubah nama *field* nomorhp menjadi no_hp dengan tipe data VARCHAR(20) pada tb_anggota.

Source Code :

```
ALTER TABLE tb_anggota CHANGE nomorhp no_hp VARCHAR(20);
```

```
DESCRIBE tb_anggota;
```

Hasil :



Gambar 6. 11 Struktur tb_anggota setelah *field* diubah menjadi no_hp

12. Ubah status salah satu peminjaman menjadi Dikembalikan dan isi tanggal kembali dengan tanggal hari ini.
    Source Code :

```
UPDATE tb_peminjaman
SET status = 'Dikembalikan', tgl_kembali = CURDATE()
WHERE id_pinjam = 1;

SELECT * FROM tb_peminjaman;
```

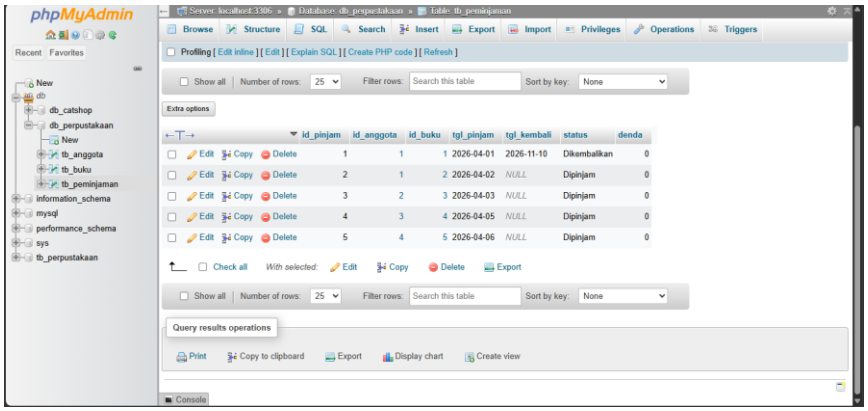
Hasil :

13. Tambahkan field denda (INT, default 0) setelah field status pada tb_peminjaman.
    Source Code :

```
ALTER TABLE tb_anggota CHANGE nomorhp no_hp VARCHAR(20);

DESCRIBE tb_anggota;
```

Hasil :



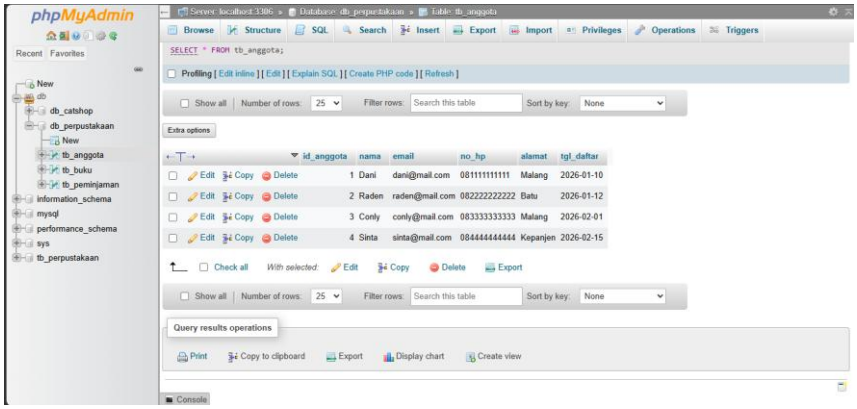
Gambar 6. 12 Hasil *UPDATE* status peminjaman

14. Hapus anggota yang belum pernah meminjam buku (id_anggota 5). Perintah berhasil dijalankan.
Source Code :

```
ALTER TABLE tb_anggota CHANGE nomorhp no_hp VARCHAR(20);

DESCRIBE tb_anggota;
```

Hasil :

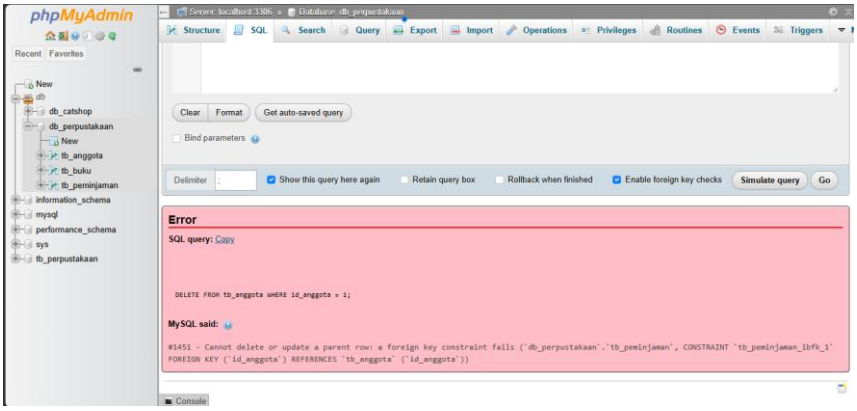


Gambar 6. 13 Hasil *DELETE* anggota yang belum meminjam

15. Hapus anggota yang sedang meminjam buku (id_anggota 1). Perintah ditolak dan muncul pesan error.
Source Code :

```
DELETE FROM tb_anggota WHERE id_anggota = 1;
```

Hasil :

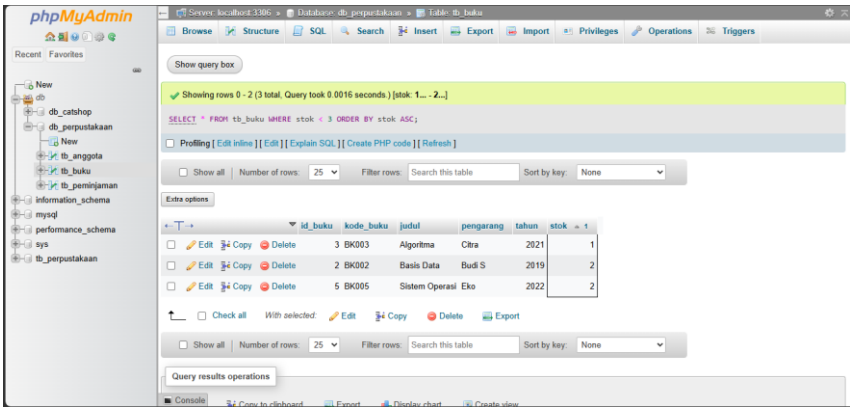


Gambar 6. 14 Pesan error saat menghapus anggota yang sedang meminjam

16. Tampilkan buku dengan stok kurang dari 3, diurutkan dari stok terkecil.
Source Code :

```
SELECT * FROM tb_buku
WHERE stok < 3
ORDER BY stok ASC;
```

Hasil :



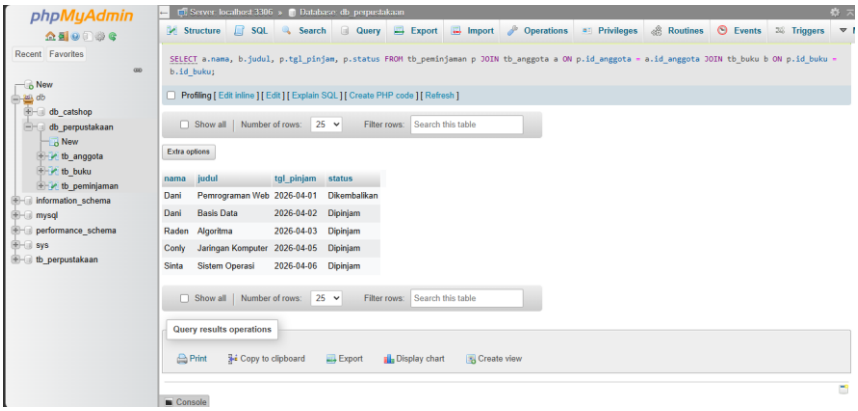
Gambar 6. 15 Hasil *query* buku dengan stok kurang dari 3

17. Tampilkan nama anggota, judul buku, tgl_pinjam, dan status dengan JOIN ketiga tabel.

Source Code :

```
SELECT a.nama, b.judul, p.tgl_pinjam, p.status
FROM tb_peminjaman p
JOIN tb_anggota a ON p.id_anggota = a.id_anggota
JOIN tb_buku b ON p.id_buku = b.id_buku;
```

Hasil :



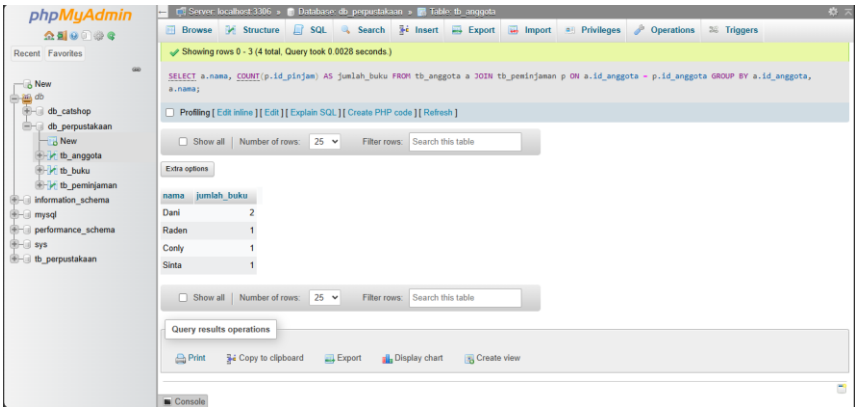
Gambar 6. 16 Hasil query JOIN tiga tabel

18. Tampilkan jumlah buku yang dipinjam oleh setiap anggota.

Source Code :

```
SELECT a.nama, COUNT(p.id_pinjam) AS jumlah_buku
FROM tb_anggota a
JOIN tb_peminjaman p ON a.id_anggota = p.id_anggota
GROUP BY a.id_anggota, a.nama;
```

Hasil :



Gambar 6. 17 Hasil query jumlah buku per anggota

Analisa:

Pada praktikum ini dibuat database db_perpustakaan yang berisi tiga tabel, yaitu tb_anggota, tb_buku, dan tb_peminjaman. Tabel induk (tb_anggota dan tb_buku) harus dibuat lebih dulu, karena tb_peminjaman memiliki dua *foreign key* yang merujuk ke kedua tabel tersebut. Pengisian data dilakukan dengan dua cara: INSERT satu per satu pada tb_anggota, dan satu perintah INSERT untuk banyak baris pada tb_buku dan tb_peminjaman, yang lebih ringkas dengan hasil yang sama. Struktur tabel kemudian diubah dengan ALTER TABLE, yaitu menambah kolom denda (default 0, sehingga lima data lama otomatis bernilai 0) dan mengganti nama kolom nomorhp menjadi no_hp dengan tipe VARCHAR(20) tanpa menghilangkan data yang sudah ada. Perintah UPDATE dengan klausa WHERE id_pinjam = 1 mengubah satu peminjaman menjadi "Dikembalikan" dan mengisi tgl_kembali dengan CURDATE(), sedangkan empat baris lainnya tidak berubah.

Pada perintah DELETE, anggota dengan id_anggota = 5 berhasil dihapus karena belum pernah meminjam buku, sehingga tidak ada data lain yang bergantung padanya. Sebaliknya, penghapusan anggota dengan id_anggota = 1 ditolak dengan error #1451 - Cannot delete or update a parent row: a foreign key constraint fails. Anggota tersebut masih memiliki data peminjaman, dan kolom id_anggota di tb_peminjaman merujuk ke tb_anggota melalui *constraint* tb_peminjaman_ibfk_1. Penolakan ini menjaga integritas referensial, yaitu aturan bahwa data yang masih dirujuk tabel lain tidak boleh hilang. Agar anggota tersebut bisa dihapus, data peminjamannya harus dihapus terlebih dahulu.

Pada tahap penampilan data, query SELECT dengan WHERE stok < 3 dan ORDER BY stok ASC menghasilkan tiga buku dengan stok paling sedikit, yaitu Algoritma, Basis Data, dan Sistem Operasi. Query JOIN pada tiga tabel mengubah angka id_anggota dan id_buku di tb_peminjaman menjadi nama anggota dan judul buku, sehingga lima data peminjaman lebih mudah dibaca. Query GROUP BY dengan COUNT menunjukkan jumlah buku yang dipinjam tiap anggota, yaitu Dani 2 buku dan Raden, Conly, serta Sinta masing-masing 1 buku. Dari seluruh langkah dapat disimpulkan bahwa MySQL tidak hanya menyimpan data, tetapi juga menjaga konsistensinya lewat *primary key*, *unique*, *not null*, dan *foreign key*,

sementara perintah CREATE, ALTER, INSERT, UPDATE, DELETE, SELECT, dan JOIN dipakai untuk membentuk, mengelola, dan mengolah data tersebut.